Interfaces Gráfricas

Juego UNO

Equipo
Daniel Paul De León Lozano
Rosa Itsel Jaramillo Garcia
Dalton Vega Juarez

Profesor: Alberto Salvador Nuñez Varela

Integrante	Nombre Github
Paul	EnigmaF17
Rosa	Itseljaramillo
Dalton	teknorexx

Repositorio: github.com/Enigmaf17/Proyecto-interfaces

Fecha de entrega: 12/10/26

Descripción general del proyecto

El proyecto es el juego de cartas UNO para tres jugadores, programado en C# con interfaz gráfica en Windows Forms y siguiendo las reglas oficiales. En esta versión el juego corre en una sola computadora y una persona controla a los tres jugadores, por eso las tres manos se muestran boca arriba. Cada partida, su resultado y todos los movimientos se guardan en una base de datos MySQL a través de una API desarrollada en Python con FastAPI.

Requisitos cubiertos

Requisito	Cómo se cumple
Juego UNO con reglas oficiales	Mazo de 108 cartas, efectos de Salta, Reversa, +2, comodín y comodín +4, castigo por no decir UNO y robo de una carta por turno
Interfaz gráfica con imágenes	Mesa circular en Windows Forms con 55 imágenes de cartas hechas por el equipo
Escrito en C#	Lógica, interfaz y pruebas en C# con .NET 10
Un jugador por integrante	Tres jugadores: Paul, Rosa y Dalton
Nombre o id de los jugadores en la base	Tabla jugadores, cargada por el script de la base de datos
Historial de partidas ganadas y perdidas	Tablas partidas y partida_jugador, consultables desde el botón Historial del juego
Log del juego	Tabla movimientos: cada jugada, robo, "UNO", castigo y paso de turno, con su número de jugada y fecha

Arquitectura

La solución de Visual Studio tiene tres proyectos de C#, y además el repositorio contiene la API y el script de la base de datos:

Componente	Tecnología	Responsabilidad
UnoLogica	Biblioteca de clases C#	Reglas del juego (Carta, Mazo, Jugador, Partida), comunicación con la API

Componente	Tecnología	Responsabilidad
		(ApiCliente) y unión de ambas (ControladorPartida)
UnoUI	Windows Forms	Mesa de juego, pantalla de carga, selección de color e historial
UnoPruebas	xUnit	Pruebas automáticas de las reglas y de la conexión con la API
api	Python, FastAPI, uvicorn	Recibe los datos del juego por HTTP y los guarda en MySQL
database	MySQL	Script que crea la base de datos y registra a los jugadores

El juego nunca se conecta directamente a MySQL. La interfaz llama a ControladorPartida, que aplica cada regla en Partida y guarda el movimiento mediante ApiCliente, que a su vez envía los datos a la API en formato JSON. Separar la lógica de la interfaz permite probar las reglas sin abrir la ventana y reutilizarlas en la siguiente versión del juego, que será en red con un esquema host-guest.

Endpoints de la API

La API corre en http://127.0.0.1:8000 y recibe y devuelve los datos en formato JSON. En el juego, la clase ApiCliente es la única que llama a estos endpoints. Mientras la API está encendida, la documentación interactiva de todos ellos se puede consultar en /docs.

Método	Ruta	Descripción
GET	/jugadores	Devuelve la lista de jugadores registrados con su id y su nombre.
POST	/partidas	Crea una partida nueva con su fecha de inicio y devuelve su id.
POST	/partidas/{id}/movimientos	Guarda un movimiento en el log de la partida: jugador, número de jugada, acción, carta y color elegido.
GET	/partidas/{id}/movimientos	Devuelve el log completo de una partida.

PUT	/partidas/{id}/finalizar	Guarda la fecha de fin y el ganador, y registra si cada jugador ganó o perdió la partida.
GET	/jugadores/{id}/estadisticas	Devuelve cuántas partidas ganó y perdió un jugador. Lo usa la ventana de historial.

Reglas implementadas y decisiones de diseño

- Se reparten 7 cartas a cada jugador. Una carta se puede jugar si coincide en color, número o tipo con la carta de arriba, o si es comodín.
- Salta hace perder el turno al siguiente jugador; Reversa cambia el sentido del juego; +2 y +4 hacen que el siguiente jugador robe y pierda su turno; los comodines permiten elegir el nuevo color.
- El jugador debe decir "UNO" antes de tirar su penúltima carta. Si no lo hace, roba dos cartas de castigo.
- Al robar se toma una sola carta. Si se puede jugar, el jugador decide si la tira; si no, pasa el turno (regla oficial).
- Cuando el mazo se acaba, se rellena con la pila de descarte, menos la carta de arriba, y se baraja.
- Gana el primer jugador que se queda sin cartas.
- Como simplificación, la primera carta de la mesa siempre es de número, y el comodín +4 se puede jugar en cualquier momento.

Ejecución

Para ejecutar el proyecto se crea la base de datos con database/schema.sql, se instalan las dependencias de la API con pip install -r requirements.txt, se copia config.example.py como config.py con la contraseña local de MySQL, se inicia la API con uvicorn main:app --reload y se ejecuta el proyecto UnoUI desde Visual Studio. El archivo README del repositorio describe cada paso.

División de tareas

Cada integrante trabajó en sus propias ramas y sus cambios entraron a la rama principal (master) mediante Pull Requests revisados por el administrador del repositorio.

Integrante	Actividades	Ramas
Paul Lozano Administrador del repositorio	Estructura de la solución y firmas de las clases. Proyecto de pruebas UnoPruebas. Reglas del juego en la clase Partida (turnos, efectos,	reglas, api, controlador, imagenes, historial

Integrante	Actividades	Ramas
	UNO, inicio de partida y robo) con sus pruebas. API en FastAPI con seis endpoints. Clases ApiCliente y ControladorPartida. Diseño de las 55 imágenes de las cartas. Pantalla de carga, ícono de la aplicación y ventana de historial de partidas con su botón en la mesa. Revisión y fusión de los Pull Requests.	
Rosa Jaramillo	Interfaz gráfica: mesa circular, manos de los tres jugadores, jugar cartas, robar, selección de color, botón UNO y aviso del ganador. Rediseño visual de la mesa. Conexión de la interfaz con la base de datos mediante ControladorPartida usando programación asíncrona. Configuración de las imágenes en el proyecto. README con las instrucciones del proyecto.	interfaz, documentacion
Dalton Vega	Clases Carta, Mazo y Jugador: creación de las 108 cartas, barajado con el algoritmo Fisher-Yates, robo y relleno del mazo, con sus pruebas automáticas. Script de la base de datos (schema.sql). Diagrama entidad-relación.	cartas-mazo, base-datos

Relación entre actividades y commits

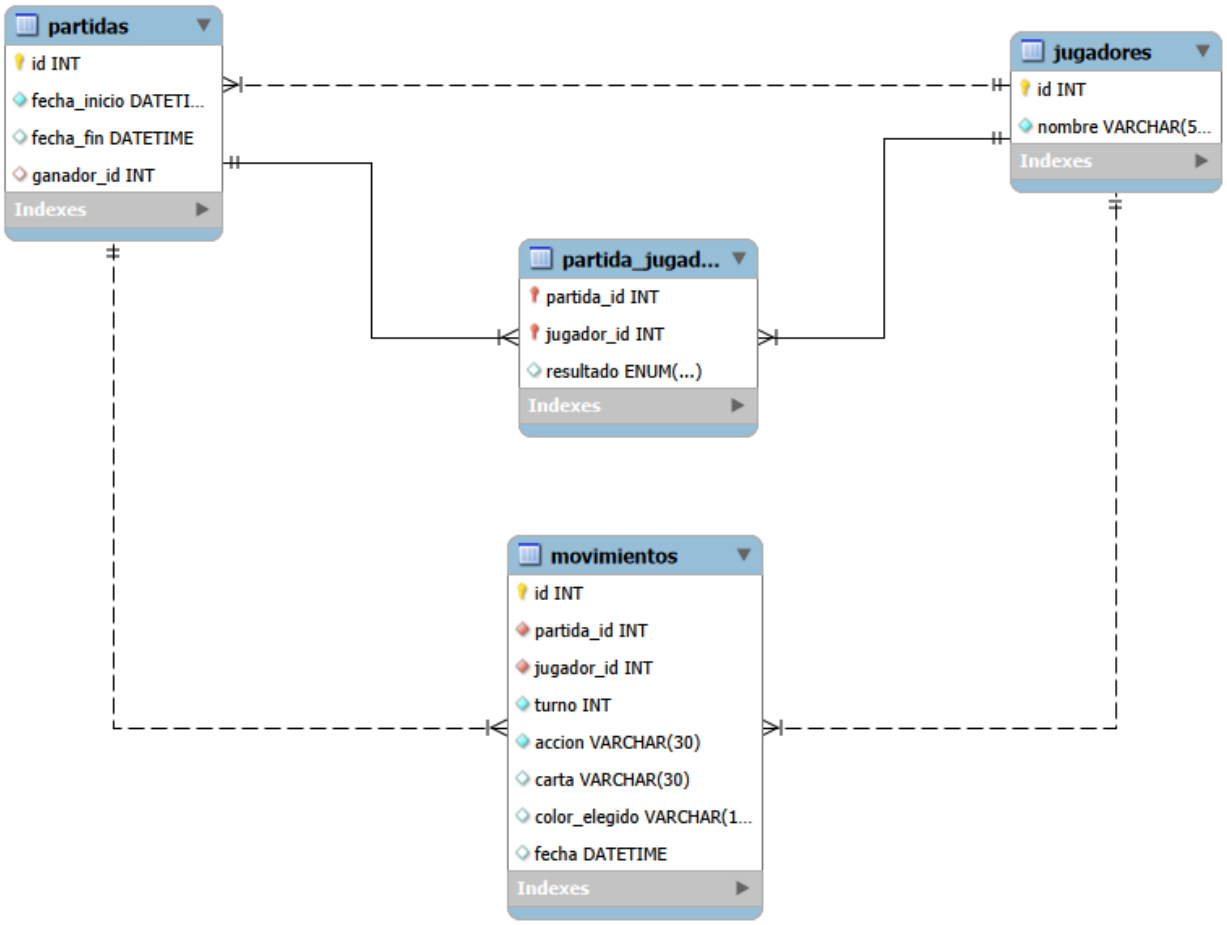
La siguiente tabla relaciona cada actividad de la división de tareas con los commits que la implementan. Los hashes son los mismos del listado de commits.

Integrante	Actividad	Commits
Paul	Estructura de la solución y firmas de las	fb515ae, 9f2ddcf, a342cc9

	clases	
	Proyecto de pruebas UnoPruebas	b3f64e6
	Reglas del juego en la clase Partida y sus pruebas	d0c83e3, e07a306, 71b8f71, e552e60, bb677e8, 547647d, 8b11c43
	API en FastAPI con seis endpoints	f534127, 310d681, 124a979, 6128e7b, 7d55bd9
	Clase ApiCliente	2d400c7
	Clase ControladorPartida y sus pruebas	20166d9, 0b404ea
	Imágenes de las cartas	e85f878
	Ícono de la aplicación y pantalla de carga	a8540ea, 132ec39
	Ventana de historial de partidas y su botón en la mesa	217414e, 07be9ae
	Correcciones durante la integración	5940a7c, 4c44cc8
	Revisión y fusión de los Pull Requests	Registrada en la pestaña Pull requests de GitHub
Rosa	Configuración de las imágenes en el proyecto	d694c34
	README con las instrucciones del proyecto	d9c846a
	Mesa del juego y manos de los jugadores	f41bf38
	Color actual, turno al centro y colores de la mesa	a836abf
	Ventana para elegir el color del comodín	c603487
	Rediseño visual estilo mesa de casino	e467193
	Conexión de la interfaz con la base de datos	c5bfe1b
Dalton	Clases Carta y Jugador	7d70370, f477cbc
	Clase Mazo: crear, barajar, robar y rellenar	1c8e2c1, df911ea, b762f5e

	Pruebas de Carta y Mazo	dc0cbdc
	Script de la base de datos (schema.sql)	1a34bf9
	Diagrama entidad-relación	64ac6be

Diagrama entidad-relación



La base de datos uno tiene cuatro tablas:

Tabla	Campos	Relaciones
jugadores	id (PK), nombre	Un jugador participa en muchas partidas y registra

Tabla	Campos	Relaciones
		muchos movimientos
partidas	id (PK), fecha_inicio, fecha_fin, ganador_id (FK)	ganador_id apunta a jugadores; una partida tiene muchos movimientos
partida_jugador	partida_id (PK, FK), jugador_id (PK, FK), resultado	Tabla intermedia de la relación muchos a muchos entre partidas y jugadores; resultado indica si la partida fue ganada o perdida
movimientos	id (PK), partida_id (FK), jugador_id (FK), turno, accion, carta, color_elegido, fecha	Cada movimiento pertenece a una partida y a un jugador

Listado de commits

Commits del repositorio desde el 21 de septiembre de 2026 hasta un día antes de la entrega. En total son 41 commits: 26 de Paul Lozano(EnigmaF17), 8 de Dalton (TeknoRexx) y 7 de Rosa (Rosa e itseljaramillo-arch).

Hash	Autor	Fecha	Mensaje
fb515ae	Paul	2026-09-23	Add .gitattributes and .gitignore.
9f2ddcf	Paul	2026-09-23	Add project files.
a342cc9	Paul	2026-10-03	Creacion de metodos
b3f64e6	Paul	2026-10-04	Proyecto de pruebas
d0c83e3	Paul	2026-10-04	Pruebas de PasarTurno y PuedeJugar
e07a306	Paul	2026-10-04	JugarCarta con efectos de cartas especiales
71b8f71	Paul	2026-10-04	DecirUno y castigo por no decirlo
e552e60	Paul	2026-10-04	Test Partida

f534127	Paul	2026-10-04	API base con FastAPI y GET /jugadores
310d681	Paul	2026-10-04	Endpoint POST /partidas
124a979	Paul	2026-10-04	Endpoints de movimientos
6128e7b	Paul	2026-10-04	Endpoint finalizar partida
7d55bd9	Paul	2026-10-04	Endpoint de estadisticas por jugador
2d400c7	Paul	2026-10-04	ApiCliente para comunicar el juego con la API
e85f878	Paul	2026-10-04	Imagenes de las cartas
7d70370	TeknoRexx	2026-10-04	Implementar Carta.NombreImagen
f477cbc	TeknoRexx	2026-10-04	Implementar metodos de Jugador
1c8e2c1	TeknoRexx	2026-10-04	Implementar Mazo.Crear
df911ea	TeknoRexx	2026-10-04	Implementar Mazo.Barajar
b762f5e	TeknoRexx	2026-10-05	Implementar Mazo.Robar y Mazo.Rellenar
dc0cbdc	TeknoRexx	2026-10-05	Agregar pruebas de Carta y Mazo
d694c34	itseljaramillo-arch	2026-10-05	Configurar imagenes para copiarse al compilar
bb677e8	Paul	2026-10-05	Iniciar: repartir cartas y voltear la primera

547647d	Paul	2026-10-05	RobarCarta con relleno del mazo
8b11c43	Paul	2026-10-05	Pruebas de Iniciar, RobarCarta, +2, +4 y castigo de UNO
20166d9	Paul	2026-10-05	ControladorPartida: une reglas y registro en la base de datos
0b404ea	Paul	2026-10-05	Pruebas de ControladorPartida
1a34bf9	TeknoRexx	2026-10-05	Agregar schema.sql de la base de datos uno
64ac6be	TeknoRexx	2026-10-05	Agregar diagrama entidad-relacion
5940a7c	Paul	2026-10-06	correcion
4c44cc8	Paul	2026-10-06	hola
d9c846a	Rosa	2026-10-06	README con instrucciones del proyecto
f41bf38	Rosa	2026-10-06	Mesa del juego y manos de los jugadores
a836abf	Rosa	2026-10-06	Mesa del color actual, turno al centro y colores nuevos
c603487	Rosa	2026-10-06	Ventana para elegir el color del comodín
e467193	Rosa	2026-10-06	Rediseño estilo mesa de casino
c5bfe1b	Rosa	2026-10-06	Conectar la interfaz con la base de datos
a8540ea	Paul	2026-10-06	Icono
132ec39	Paul	2026-10-06	Pantalla de carga

217414e	Paul	2026-10-10	Ventana de historial de partidas
07be9ae	Paul	2026-10-10	Botón de historial en la mesa

Listado de ramas



Rama	Responsable	Contenido
master	Todo el equipo	Versión estable; solo recibe cambios por Pull Request
reglas	Paul	Clase Partida y sus pruebas
api	Paul	API en Fast API y clase ApiCliente
controlador	Paul	Clase ControladorPartida y sus pruebas

Rama	Responsable	Contenido
imagenes	Paul	Imágenes de las cartas
cartas-mazo	Dalton	Clases Carta, Mazo y Jugador y sus pruebas
base-datos	Dalton	Script de la base de datos
interfaz	Rosa	Interfaz gráfica y su conexión con la base de datos
documentación	Rosa	README del proyecto
historial	Paul	Ventana del historial de partidas

Problemas encontrados y cómo se solucionaron

Proyecto general

Problema	Solución
Las reglas no se podían probar mientras no existiera la interfaz.	Se creó el proyecto UnoPruebas con xUnit para probar cada regla de forma automática, sin abrir la ventana.
Al rellenar el mazo con la pila de descarte, las cartas podían quedar repetidas en ambos lugares.	El método Rellenar quita las cartas del descarte (salvo la de arriba) antes de pasarlas al mazo, y una prueba automática lo verifica.
Registrar el log desde la interfaz obligaba a repetir en cada botón la llamada a la API con todos sus datos.	Se creó la clase ControladorPartida, que aplica cada regla y guarda el movimiento en un solo paso.
La ventana se congelaba mientras se guardaba un movimiento, y un doble clic podía jugar dos veces.	Se usó programación asíncrona (async y await) y una bandera que bloquea los clics hasta terminar de guardar.
Si la API estaba apagada, el juego no podía iniciar la partida.	Se agregó una pantalla de carga y un aviso con la opción de reintentar la conexión.

Problema	Solución
La API respondía con error 422 en algunas pruebas.	El cuerpo de la petición no tenía la forma del modelo esperado; se corrigió el formato del JSON enviado.
Las imágenes de las cartas se exportaron en formato GIF y con nombres genéricos.	Se reexportaron en PNG y se renombraron las 55 con un script de PowerShell, según la convención color_valor.png.
Al ejecutar el juego no se encontraban las imágenes.	Se configuraron con la opción Copy if newer para que se copien junto al ejecutable.
Los primeros colores de la interfaz dificultan leer el texto sobre la mesa.	Se rediseñó la mesa con fieltro verde, fondo oscuro y un anillo que indica el color actual.
El equipo no coincidía en la regla de robar (una carta o robar hasta poder jugar).	Se revisaron las reglas oficiales y se implementó el robo de una sola carta.
En PowerShell no se podía activar el entorno virtual de Python por la política de ejecución de scripts.	Se instalaron las dependencias directamente con pip; el README indica cómo cambiar la política si se quiere usar el entorno virtual.

Git y GitHub

Problema	Solución
El primer proyecto se creó en una carpeta de OneDrive, fuera del repositorio clonado, y Visual Studio mostraba "0 Repositories".	Se eliminó y se volvió a crear la solución dentro de la carpeta del repositorio, fuera de OneDrive, para evitar además conflictos de sincronización.
No se podía agregar la referencia entre proyectos.	Visual Studio estaba en la vista de carpetas (Folder View); se abrió la solución en la vista de solución.
Las ramas creadas desde GitHub no aparecían en Visual Studio.	Se hizo Fetch y se descargaron las ramas desde la pestaña Remotes.
Las gráficas de Insights, como el árbol de ramas, no estaban disponibles con el repositorio privado.	Se hizo público el repositorio.
La contraseña de MySQL podía subirse a un repositorio público.	El archivo config.py se excluyó con .gitignore y se subió config.example.py como plantilla.

Problema	Solución
Al probar el código en la computadora de otro integrante, los commits podían quedar a nombre equivocado.	Cada integrante configurar Git con el correo de su cuenta de GitHub y sube sus cambios desde su propia computadora.
Riesgo de conflictos al integrar el trabajo de los tres integrantes en los mismos archivos.	Cada archivo tuvo un solo responsable, no se cambiaron los nombres públicos de los métodos y cada integrante trajo los cambios de master a su rama antes de seguir trabajando.
Al clonar el repositorio en una carpeta nueva, Visual Studio intentaba ejecutar la biblioteca de clases.	El proyecto de inicio no se guarda en el repositorio; se documentó en el README que hay que establecer UnoUI como proyecto de inicio.